

© Joshua M. Willman 2020

J. M. Willman, *Beginning PyQt*

[https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-5857-6\\_11](https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-5857-6_11)

---

## 11. Managing Threads

Joshua M. Willman<sup>1</sup>

(1) Hampton, VA, USA

We have all experienced that moment when running some process such as copying files between directories or launching a new instance of an application causes a program to lag for just a moment and, in some cases, to freeze completely. We are then forced to either wait for the current task to complete or Ctrl+Alt+Delete our way to freedom. When you are creating GUIs, you should be aware of how to handle, or more preferably have foresight about avoiding, these situations.

The motivation behind this chapter is twofold – to help you design more robust GUI applications and to inform you of how you might be able to handle situations where your applications need to run long processes. Any action that causes event processing in an application to come to a standstill is bad for a user's experience.

This chapter takes a look at

- How to implement threading with `QThread`
- A few other techniques for handling time-consuming processes
- The `QProgressBar` widget for giving visual feedback about a task's progression

### Introduction to Threading

A computer's performance can be measured by the accuracy, efficiency, and speed at which it can execute program instructions. Modern computers can take advantage of their multicore processors to run those instructions in parallel, thereby increasing the performance of computer applications that have been written to take advantage of the multicore architecture.

The idea of performing tasks in a synchronous manner, that is, where only one task is processed at a time until completion before moving on to the next task, can be inefficient, especially for larger operations. What we need is a way to perform operations concurrently. That is where threads and processes come into play.

Threads and processes are not the same thing. Without going too much into the technical jargon, let's try and understand the differences between the two. A **process** is an instance of an application that requires memory and computer resources to run. Opening up the word processor on your computer to write an essay is one process. While writing your essay, you

also need to search on the Internet for information. You now have two separate processes running on your computer independently and in parallel. What happens in one process is not influencing the other. Of course, you have multiple tabs open in the web browser, and each tab is loading and updating information; those tabs are working side by side with the web browser. This is where a thread becomes important.

A **thread** is essential to the concurrency within an individual process. When a process begins, it only has one thread, and multiple threads can be started within a single process. These threads, just like the processes, are managed by the CPU. **Multithreading** occurs when the CPU can handle multiple threads of execution concurrently within one process. These threads are independent but also share the process's resources. Using multithreading allows for applications to be more responsive to user's inputs while other operations are occurring in the background, and to better utilize a system's resources.

On a system with only a single CPU, true parallelism is actually unachievable. In these instances, the CPU is shared among the processes or threads. To switch between threads, context switches are used to interrupt the current thread, save its state, and then restore the next thread's state. This gives the user a false appearance of parallelism.

To achieve true parallelism and create a truly concurrent system, a multi-core processor would allow threads in a multithreaded application to be assigned to different processors.

### Threading in PyQt

Applications based on Qt are event based. When the event loop is started using `exec_()`, a thread is created. This thread is referred to as the main thread of the GUI. Any events that take place in the main thread, including the GUI itself, run synchronously within the main event loop. To take advantage of threading, we need to create a secondary thread to offload processing operations from the main thread.

PyQt makes communicating between the main thread and secondary threads, also referred to as worker threads, simple with signals and slots. This can be useful for relaying feedback, allowing the user to interrupt a process, and for informing the main thread that a process has finished. Since threads utilize the same address space, they can share data very easily.

However, if multiple threads try to access shared data or resources concurrently, this can cause crashes or memory corruption. Deadlock is another issue that can occur if two threads are blocked because they are waiting for resources. PyQt provides a few classes, for example, `QMutex`, `QReadWriteLock`, and `QSemaphore`, for avoiding these kinds of problems.

**Note** Python also has a number of modules for handling threading and processing tasks, including `_thread`, `threading`, `asyncio`, and `multiprocessing`. While you can also use these modules, PyQt's `QThread` and other classes allow you to emit signals between the main and worker threads.

## Methods for Processing Long Events in PyQt

While this chapter focuses on using `QThread`, it is also a good idea to keep in mind that there are also other ways that you might want to try before attempting to use threading in your GUI. Implementing threading can lead to problems with concurrency and identifying errors. Combined with signals and slots, PyQt provides a few different ways to handle time-consuming operations.

Choosing which method is best for your application comes down to considering your situation. The main methods, including threading, for handling these kinds of events are listed as follows:

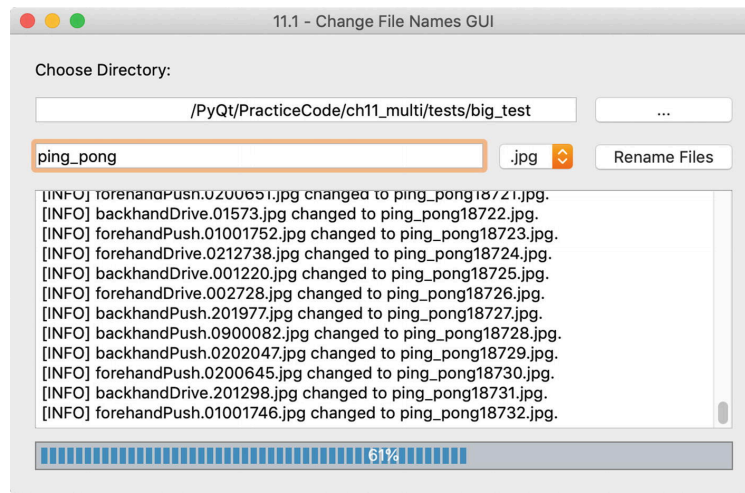
1. If there is a process in your application that is causing it to freeze, check to see if that process can be broken down into smaller steps and perform them sequentially. Manually handle the processing of long operations, and explicitly call `QApplication.processEvents()` to process pending events. This works best if your operations can be processed using a single thread.
2. With `QTimer` and signals and slots, you can schedule operations to be performed at certain intervals in the future.
3. Use `QThread` to create a worker thread that will perform long operations in a separate thread. Derive a class from `QThread`, reimplement `run()`, and use PyQt's signal and slot mechanism to communicate with the main thread. This method can help to avoid blocking the main event loop.
4. The `QThreadPool` and `QRunnable` classes can be used to divide the work across the CPUs on your computer. Create a subclass of `QRunnable` and reimplement the `run()` function; an instance of `QRunnable` can then be passed to threads that are managed by `QThreadPool`. `QThreadPool` handles the queuing and execution of `QRunnable` instances for you.

There are even other options that may depend upon your application's requirements. Keep in mind that, while using threads could benefit your application, they could also slow it down or cause errors if used incorrectly.

## Project 11.1 – File Renaming GUI

This chapter's project, shown in Figure 11-1, actually stems from my own experiences. Creating datasets for training neural networks often entails writing Python scripts for labeling thousands of images and data files. Those scripts are generally written to include some kind of visual feedback to the user about how the process is going in the command line.

For this project, we are going to create a GUI that will allow us to select a local directory and edit the names of files with the specified extension. The interface includes `QTextEdit` and `QProgressBar` widgets as two different means of feedback about the file labeling process. This application also takes advantage of the `QThread` class so that users are still able to interact with the interface while the operations are being performed in the background.



**Figure 11-1** The interface for renaming files in a selected directory

## The QProgressBar Widget

The QProgressBar widget visually relays the progress of an extended operation back to the user. This feedback can also be used as reassurance that a process, such as a download, installation, or file transfer, is still running. Some of the settings that can be controlled include the widget's orientation and range.

Refer to the project in this chapter for setting up the progress bar.

## File Renaming GUI Solution

The GUI window contains various buttons and editor widgets that allow the user to manage file renaming. The user can select a directory using a QFileDialog. They can also enter the new file name in the QLineEdit widget. Using the combo box, they can select the file extension for the files they want to change.

The application uses threading to update the progress bar and display information about the files being changed in the text edit and performs the actual renaming operation. This is all done using signals and slots. The code for the file renaming application can be found in Listing 11-1.

```
# file_rename_threading.py
import os, sys, time
from PyQt5.QtWidgets import (QApplication, QWidget, QLabel, QProgressBar, QLineEdit, QPushButton, QText
from PyQt5.QtCore import pyqtSignal, QThread
style_sheet = """
    QProgressBar{
        background-color: #C0C6CA;
        color: #FFFFFF;
        border: 1px solid grey;
        padding: 3px;
        height: 15px;
        text-align: center;
    }
    QProgressBar::chunk{
        background: #538DB8;
        width: 5px;
        margin: 0.5px
    }
}
```

```

"""
# Create worker thread for running tasks like updating the progress bar, renaming photos,
# displaying information in the text edit widget
class Worker(QThread):
    updateValueSignal = pyqtSignal(int)
    updateTextEditSignal = pyqtSignal(str, str)
    def __init__(self, dir, ext, prefix):
        super().__init__()
        self.dir = dir
        self.ext = ext
        self.prefix = prefix
    def run(self):
        """
        The thread begins running from here. run() is only called after start().
        """
        for (i, file) in enumerate(os.listdir(self.dir)):
            _, file_ext = os.path.splitext(file)
            if file_ext == self.ext:
                new_file_name = self.prefix + str(i) + self.ext
                src_path = os.path.join(self.dir, file)
                dst_path = os.path.join(self.dir, new_file_name)
                # os.rename(src, dst): src is original address of file to be renamed
                # and dst is destination location with new name.
                os.rename(src_path, dst_path)
                #time.sleep(0.2) # Uncomment if process is too fast and want to see

the updates.
                self.updateValueSignal.emit(i + 1)
                self.updateTextEditSignal.emit(file, new_file_name)
            else:
                pass
        self.updateValueSignal.emit(0) # Reset the value of the progress bar
class RenameFilesGUI(QWidget):
    def __init__(self):
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen.
        """
        self.setMinimumSize(600, 250)
        self.setWindowTitle('11.1 - Change File Names GUI')
        self.directory = ""
        self.cb_value = ""
        self.setupWidgets()
        self.show()
    def setupWidgets(self):
        """
        Set up the widgets and layouts for interface.
        """
        dir_label = QLabel("Choose Directory:")
        self.dir_line_edit = QLineEdit()
        dir_button = QPushButton('...')
        dir_button.setToolTip("Select file directory.")
        dir_button.clicked.connect(self.setDirectory)
        self.change_name_edit = QLineEdit()
        self.change_name_edit.setToolTip("Files will be appended with numerical values. For example: fi
self.change_name_edit.setPlaceholderText("Change file names to...")
        rename_button= QPushButton("Rename Files")

```

```

rename_button.setToolTip("Begin renaming files in directory.")
rename_button.clicked.connect(self.renameFiles)
file_exts = [".jpg", ".jpeg", ".png", ".gif", ".txt"]

# Create combo box for selecting file extensions.
ext_cb = QComboBox()
self.cb_value = file_exts[0]
ext_cb.setToolTip("Only files with this extension will be changed.")
ext_cb.addItem(file_exts)
ext_cb.currentTextChanged.connect(self.updateCbValue)
# Text edit is for displaying the file names as they are updated.
self.display_files_edit = QTextEdit()
self.display_files_edit.setReadOnly(True)
self.progress_bar = QProgressBar()
self.progress_bar.setValue(0)
# Set layout and widgets.
grid = QGridLayout()
grid.addWidget(dir_label, 0, 0)
grid.addWidget(self.dir_line_edit, 1, 0, 1, 2)
grid.addWidget(dir_button, 1, 2)
grid.addWidget(self.change_name_edit, 2, 0)
grid.addWidget(ext_cb, 2, 1)
grid.addWidget(rename_button, 2, 2)
grid.addWidget(self.display_files_edit, 3, 0, 1, 3)
grid.addWidget(self.progress_bar, 4, 0, 1, 3)
self.setLayout(grid)
def setDirectory(self):
    """
    Choose the directory.
    """
    file_dialog = QFileDialog(self)
    file_dialog.setFileMode(QFileDialog.Directory)
    self.directory = file_dialog.getExistingDirectory(self, "Open Directory", "", QFileDialog.ShowD
if self.directory:
    self.dir_line_edit.setText(self.directory)
    # Set the max value of progress bar equal to max number of files in the directory.
    num_of_files = len([name for name in os.listdir(self.directory)])
    self.progress_bar.setRange(0, num_of_files)
def updateCbValue(self, text):
    """
    Change the combo box value

    . Values represent the different file extensions.
    """
    self.cb_value = text
def renameFiles(self):
    """
    Create instance of worker thread to handle the file renaming process.
    """
    prefix_text = self.change_name_edit.text()
    if self.directory != "" and prefix_text != "":
        self.worker = Worker(self.directory, self.cb_value, prefix_text)
        self.worker.updateValueSignal.connect(self.updateProgressBar)
        self.worker.updateTextEditSignal.connect(self.updateTextEdit)
        self.worker.start()
    else:
        pass

```

```

def updateProgressBar(self, value):
    self.progress_bar.setValue(value)
def updateTextEdit(self, old_text, new_text):
    self.display_files_edit.append("[INFO] {} changed to {}".format(old_text, new_text))
if __name__ == "__main__":
    app = QApplication(sys.argv)
    app.setStyleSheet(style_sheet)
    window = RenameFilesGUI()
    sys.exit(app.exec_())

```

*Listing 11-1* Code for the GUI that renames files in a directory using threading

The application's GUI can be seen in Figure [11-1](#).

## Explanation

We start with importing Python and PyQt classes. The style sheet is used to modify the appearance of the QProgressBar.

Let's start by looking at the `RenameFileGUI` class. Here we set up the window and other widgets, including push buttons for selecting the directory and starting the process for renaming files, line edit widgets, and the text edit and the progress bar widgets for relaying feedback.

The user can select a directory using `QFileDialog`. Once a directory is chosen, the user can enter the new file names into `change_name_edit` and select the file extension for the types of files to change in the combo box.

Renaming the files could take place in the main thread. This wouldn't be a problem for a few files. However, if the user wants to work with a large number of files, this would cause the GUI to be locked until the operations are finished. Therefore, the process for renaming the files, along with updating the progress bar and the text edit widgets, is performed in the worker thread.

For this project, we subclass `QThread`. An instance of the `QThread` class manages only one thread. Two custom signals are created for updating the progress bar and text edit widgets.

```

updateValueSignal = pyqtSignal(int)
updateTextEditSignal = pyqtSignal(str, str)

```

The reimplemented `QThread` method `run()` begins executing the thread. The time-consuming operations – traversing the directory, renaming files, and emitting the signals for updating the `QProgressBar` and `QTextEdit` – are performed in `run()`. However, this method is not called directly. The `QThread` method `start()` is used to communicate with the worker thread and begin executing the thread by calling `run()`. The `start()` method is called in `renameFiles()`.

## Summary

Preventing GUIs from becoming frozen while processing long operations is important for a user's experience. There are a few options for effec-

tively handling blocking in your application, including using timers and threads. PyQt makes using threading seem relatively simple with QThread and the signal and slot mechanism. However, you must be careful when using QThread to ensure that threads protect access to their own data. While not displayed in this chapter's short project, QThread also has methods, such as `started()`, `finished()`, `wait()`, and `quit()`, for managing threads.

In Chapter [12](#), we will take a look at an array of projects that utilize different PyQt classes.